ACTIVIDAD DE COMPROBACIÓN N.° 2 (AC2) · INSTRUCCIONES PARA ESTUDIANTES

Mi primera aplicación en C

Núcleo Comercio y Servicios · Subsector Informática y Comunicación

Módulo	CSTI12010 Diseño de algoritmos
Unidad de aprendizaje	UA2 · Empleo de lenguajes de programación en el paradigma de programación estructurada
Tipo de evaluación	Por producto · individual · requisito para la UA3
Fecha y duración	Martes 20 de octubre de 2026 · 4 horas (1.ª oportunidad)
Herramientas	VS Code, gcc (MSYS2 UCRT64) y git
Persona facilitadora	Giovanni Antonio Coto Calderón
Persona estudiante	Valery Shanay Mora Acevedo

1. Propósito de la actividad

Esta actividad evalúa su capacidad de **emplear los elementos del lenguaje C y sus estructuras de control** para resolver problemas empresariales. Los problemas son los mismos tipos de la AC1, ahora programados en C y con **arreglos** y **cadenas**.

2. Instrucciones generales

- Trabaje de forma **individual**. Lea cada problema completo antes de programar.
- Resuelva **los tres problemas**. Dispone de **4 horas**; los tiempos estimados suman 3 h 10 min y el resto es para el README, las pruebas y la entrega.
- Cada programa debe **compilar con gcc -Wall -Wextra sin errores ni advertencias** y producir la salida esperada con los datos del ejemplo.
- Escriba todo el programa dentro de **main**. Puede usar las funciones de biblioteca vistas (stdio.h y string.h).
- Los mensajes de printf van **sin tildes ni ñ**.
- Haga **commits durante la actividad** (al menos uno por problema) y un **push** al final.
- Sea honesto en su trabajo; no se acepta plagio.

Antes de empezar (5 minutos)

1. Abra Git Bash en su repositorio: **cd /c/INA/<su-repositorio>** y ejecute **git pull**.
2. Cree la carpeta: **mkdir -p UA2/AC2** y ábrala en VS Code.
3. Compile cada programa así: **gcc -Wall -Wextra AC2_P1_Nomina.c -o AC2_P1_Nomina** y ejecútelo con **./AC2_P1_Nomina**.

3. Qué se entrega

Archivo (carpeta UA2/AC2/)	Contenido
`README.md`	Su nombre, la fecha y, por cada problema: el análisis (datos de entrada, proceso, datos de salida y limitaciones) y una captura de la ejecución con los datos del ejemplo. Use la plantilla README_AC2_plantilla.md.
`AC2_P1_Nomina.c`	Programa del problema 1.
`AC2_P2_Calificaciones.c`	Programa del problema 2.
`AC2_P3_Compras.c`	Programa del problema 3.

4. Problema 1 · Sistema de cálculo de nómina

Dificultad: básica | **Tiempo estimado:** 50 minutos | **Archivo:** AC2_P1_Nomina.c

Una empresa de consultoría necesita calcular el salario neto de sus empleados. Escriba un programa en C que:

1. Reciba el **nombre completo** del empleado (puede tener espacios; no se acepta un nombre vacío).
2. Reciba el **salario bruto**, que debe ser **mayor que 0**; si no, muestra un mensaje y lo vuelve a pedir.
3. Calcule los descuentos sobre el salario bruto: **CAJA** (9.5 %), **IESS** (2.5 %) e **impuesto sobre la renta** (10 %, solo si el salario bruto **supera 800**).
4. Calcule el **salario neto** (bruto menos todos los descuentos).
5. Muestre un comprobante con el nombre, el salario bruto, cada descuento y el salario neto, con **dos decimales** y los montos alineados.

> **Requisitos en C**
>
> Use **constantes** (#define o const) para las tasas y para el límite de 800, **fgets** para el nombre, **strlen** para revisar que no esté vacío y un **do … while** para validar el salario.
>
> Al inicio del archivo escriba en comentarios el **enunciado**, **qué se logra**, **cómo se logra** y el **resultado esperado**, y comente el uso de cada variable y constante.

a) Análisis (en el README, con esta plantilla):

Datos de entrada	Proceso	Datos de salida	Limitaciones
1.Nombre completo del empleado (cadena con espacios, ej: Maria Lopez) 2.Salario bruto (flotante > 0, ej: 1000)	1. Validar que el nombre no esté vacío usando fgets y strlen. 2. Validar que el salario sea mayor a 0 con do-while. 3. Calcular descuentos: CAJA (9.5%), IESS (2.5%) y Renta (10% si bruto > 800). 4. Calcular salario neto restando los descuentos al bruto.	1.Nombre del empleado 2.Salario bruto 3.Descuento CAJA 4.Descuento IESS 5.Impuesto sobre la renta 6.alario neto (con 2 decimal -es y montos alineados)	1.El salario bruto debe ingresarse estrictamente en formato numérico. 2.Nombres sin tildes ni ñ por restricciones de consola

b) Programa en C: compílelo sin mensajes y pruébelo con los datos del ejemplo, con un **caso límite** y con un **dato inválido**.

Ejemplo de entrada y salida esperada · datos: Maria Lopez, 1000

```
=== Calculo de nomina ===
Nombre del empleado: Maria Lopez
Salario bruto: 1000

------- Comprobante de pago -------
Empleado:                Maria Lopez
Salario bruto:              1000.00
Descuento CAJA (9.5%):        95.00
Descuento IESS (2.5%):        25.00
Impuesto renta (10%):        100.00
Salario neto:                780.00
```

5. Problema 2 · Validación de calificaciones estudiantiles

Dificultad: intermedia | **Tiempo estimado:** 60 minutos | **Archivo:** AC2_P2_Calificaciones.c

Un centro educativo necesita automatizar el cálculo de notas finales. Escriba un programa en C que:

1. Reciba el **nombre completo** del estudiante.
2. Reciba **tres calificaciones** de 0 a 100 y las guarde en un **arreglo**. Cada calificación fuera de rango se rechaza con un mensaje y se vuelve a pedir.
3. Calcule el **promedio** de las tres calificaciones y busque la **calificación más alta** recorriendo el arreglo.
4. Determine la condición: **APROBADO** si el promedio es 70 o más; **EN RIESGO** si es 60 o más, pero menor que 70; **NO APROBADO** si es menor que 60. Guarde la condición en una **cadena**.
5. Muestre el nombre, las tres calificaciones, el promedio (dos decimales), la calificación más alta y la condición.

Requisitos en C

Use un **arreglo** float notas[3] recorrido con **for**, un **do … while** para validar cada nota, **constantes** para los límites y **strcpy** para guardar la condición.

Al inicio del archivo escriba en comentarios el **enunciado**, **qué se logra**, **cómo se logra** y el **resultado esperado**, y comente el uso de cada variable y constante.

a) Análisis (en el README, con esta plantilla):

Datos de entrada	Proceso	Datos de salida	Limitaciones
1.Nombre completo del estudiante (ej: Juan Perez) 2.Tres calificaciones numéricas entre 0 y 100 (ej: 75, 82, 68)	1. Recibir nombre validando que no esté vacío. 2. Almacenar las 3 notas en un arreglo float notas[3] usando un bucle for y validando cada valor con do-while. 3. Recorrer el arreglo para calcular la suma, el promedio y hallar la nota más alta. 4. Asignar condición (APROBADO, EN RIESGO, NO A PROBADO) usando strcpy.)	1.Nombre del estudiante 2.Detalle de las 3 calificaciones promedio (2 decimales) 3.Nota más alta 4.Condición académica	Solo procesa exactamente tres calificaciones. Rango de notas limitado de 0 a 100.

b) Programa en C: compílelo sin mensajes y pruébelo con los datos del ejemplo, con un **caso límite** y con un **dato inválido**.

Ejemplo de entrada y salida esperada · datos: Juan Perez, 75, 82, 68

```
=== Calificaciones del estudiante ===
Nombre del estudiante: Juan Perez
Calificacion 1 (0 a 100): 75
Calificacion 2 (0 a 100): 82
Calificacion 3 (0 a 100): 68

------- Resultado -------
Estudiante: Juan Perez
Calificacion 1:   75.0
Calificacion 2:   82.0
Calificacion 3:   68.0
Promedio:        75.00
Nota mas alta:    82.0
Condicion:      APROBADO
```

6. Problema 3 · Gestión de compras en una tienda

Dificultad: avanzada | **Tiempo estimado:** 80 minutos | **Archivo:** AC2_P3_Compras.c

Una tienda de electrónica necesita procesar las compras de sus clientes. Escriba un programa en C que:

1. Reciba el **nombre del cliente**.
2. Permita registrar **varios productos** (máximo 10): nombre (puede tener espacios), precio unitario y cantidad. Después de cada producto pregunta si desea agregar otro (**S/N**); solo acepta S o N (mayúscula o minúscula).
3. Valide que el precio y la cantidad sean **positivos**.
4. Muestre el **subtotal** de cada producto (precio × cantidad) al ingresarlo y acumule el **total**.
5. Aplique el descuento según el total: menos de 100, sin descuento; de 100 a menos de 500, **5 %**; de 500 a 1000, **10 %**; más de 1000, **15 %**.
6. Calcule el total con descuento y el **impuesto** (13 % sobre el total con descuento).
7. Muestre una **factura** con el cliente, el **detalle de cada producto** (nombre, precio, cantidad y subtotal), la cantidad de productos, el subtotal, el descuento, el total con descuento, el impuesto y el **total a pagar**.

Requisitos en C

Guarde los productos en **arreglos paralelos** (nombres, precios y cantidades) para poder mostrar el detalle al final con un **for**. Use **fgets** para los nombres. Recuerde descartar el Enter pendiente (getchar) antes de leer un nombre con fgets después de un scanf.

Al inicio del archivo escriba en comentarios el **enunciado**, **qué se logra**, **cómo se logra** y el **resultado esperado**, y comente el uso de cada variable y constante.

a) Análisis (en el README, con esta plantilla):

Datos de entrada	Proceso	Datos de salida	Limitaciones
Nombre del cliente (ej: Ana Garcia) Por cada producto (hasta 10): Nombre, Precio unitario (> 0), Cantidad (> 0) Respuesta de continuación (S/N)	1. Usar arreglos paralelos (nombres, precios, cantidades, subtotales). 2. Validar entradas con ciclos y limpiar el buffer con getchar tras scanf. 3. Calcular subtotal por producto y acumular el subtotal general. 4. Aplicar descuento por tramos (<100: 0%, 100-500: 5%, 500-1000: 10%, >1000: 15%). 5. cular impuesto del 13% sobre el total con descuento y el total final a pagar.	Factura detallada con cliente, tabla de productos, cantidad r egistrada, subtotal, descuento aplicado, total con descuento, impuesto (13%) y TOTAL A PAGAR.	Máximo de 10 productos por compra. Las entradas de control de menú solo aceptan S o N.

b) Programa en C: compílelo sin mensajes y pruébelo con los datos del ejemplo, con un **caso límite** y con un **dato inválido**.

Ejemplo de entrada y salida esperada · datos: Ana Garcia, Teclado, 60, 2, S, Mouse, 25, 1, S, Monitor, 350, 1, N

```
=== Compra en la tienda ===
Nombre del cliente: Ana Garcia

Producto 1
  Nombre: Teclado
  Precio unitario: 60
  Cantidad: 2
  Subtotal: 120.00
Desea agregar otro producto? (S/N): S

Producto 2
  Nombre: Mouse
  Precio unitario: 25
  Cantidad: 1
  Subtotal: 25.00
Desea agregar otro producto? (S/N): S

Producto 3
  Nombre: Monitor
  Precio unitario: 350
  Cantidad: 1
  Subtotal: 350.00
Desea agregar otro producto? (S/N): N

================= FACTURA =================
Cliente: Ana Garcia
Producto               Precio Cant  Subtotal
Teclado                 60.00    2    120.00
Mouse                   25.00    1     25.00
Monitor                350.00    1    350.00
Productos registrados: 3
Subtotal:                   495.00
Descuento:                   24.75 (5%)
Total con descuento:        470.25
Impuesto (13%):              61.13
TOTAL A PAGAR:              531.38
```

7. Criterios de evaluación

Aspecto	Indicador
Desempeño	✓ Orden en el código fuente: sangría, nombres descriptivos y comentarios.
	✓ Entrega dentro del tiempo de la actividad.
Repositorio y entrega	✓ Carpeta UA2/AC2 con los tres programas y el README.md. *

Aspecto	Indicador
	✓ Commits descriptivos durante la actividad y push final.
	✓ Archivo Proyecto02_PrimerNombre_PrimerApellido.zip con el contenido de la carpeta.
Análisis	✓ Datos de entrada, proceso, datos de salida y limitaciones de cada problema.
Programas en C	✓ Los tres programas compilan sin errores. *
	✓ Compilan sin advertencias.
	✓ Tipos de datos adecuados y constantes para tasas y límites.
	✓ Entrada y salida con el formato correcto (fgets para nombres, montos con 2 decimales).
	✓ Validación de datos con ciclos.
	✓ Problema 1: produce la salida esperada (salario neto 780.00). *
	✓ Problema 2: usa un arreglo y produce la salida esperada (promedio 75.00, APROBADO). *
	✓ Problema 3: usa arreglos paralelos y produce la salida esperada (TOTAL A PAGAR 531.38). *
	✓ Usa correctamente las cadenas (fgets, strcspn, strlen, strcpy; no = ni == con cadenas).
	✓ Comentarios con el enunciado, qué se logra, cómo se logra y el uso de las variables.

Los indicadores marcados con * son **críticos**: deben estar logrados. Además, como máximo 2 indicadores pueden quedar pendientes.

8. Entrega

1. **Repositorio (evidencia):** git add, git commit -m "AC2: …" y git push de la carpeta UA2/AC2 antes de que termine el tiempo.
2. **Respaldo:** comprima la carpeta AC2 en **Proyecto02_PrimerNombre_PrimerApellido.zip** (ejemplo: Proyecto02_Ana_Mora.zip) y entréguelo como archivo adjunto al correo electrónico gcotocalderon@ina.cr.

9. Notas importantes

- Un programa que no compila no cumple los indicadores de funcionamiento de ese problema.
- Si no logra el resultado Conforme, tendrá una **segunda oportunidad de 3 horas** en la fecha que indique la persona facilitadora.
- Esta actividad es **requisito** para avanzar a la Unidad de Aprendizaje 3.

Instituto Nacional de Aprendizaje (INA) · Octubre 2026